

Laporan Tugas Besar IF1221
Mata Kuliah Logika Komputasional



Kelompok 8 IF1221-K01 Logika Komputasional

- | | | |
|----|-------------------------|----------|
| 1. | Ghina Emelia Yantes | 13525119 |
| 2. | Sahla Nailah Salsabilla | 13525134 |
| 3. | Cathrine Angel Siburian | 13525138 |
| 4. | Nayla Putri Ghaisani | 13525140 |

Institut Teknologi Bandung

Mei 2026

DAFTAR ISI

DAFTAR ISI.....	2
A. PENJELASAN QUERY.....	3
a. start.....	3
b. anggota.....	5
c. panduan.....	6
d. help.....	6
e. exit.....	8
f. mainkanKartu.....	9
g. ambilKartu.....	11
h. cekInfo.....	12
i. tantang.....	14
j. uni.....	15
k. tangkap.....	16
l. lihatKartu.....	17
m. lihatCommand.....	18
n. endGame.....	19
o. saveGame.....	21
p. loadGame.....	23
q. godsHand.....	25
r. sembunyikanKartu.....	26
s. Mimic Card.....	28
t. turnamen.....	29
B. PEMBAGIAN DAN PERSENTASE KERJA ANGGOTA.....	34
FORMAT DEKLARASI PENGGUNAAN AI.....	35

A. PENJELASAN QUERY

a. start

Tahap ini merupakan proses inisialisasi permainan (startGame) yang dilakukan dengan menjalankan perintah start. Pada tahap ini program akan melakukan beberapa proses awal, yaitu inisialisasi pemain, distribusi kartu awal, serta inisialisasi discard pile.

Contoh Program
<pre> ?- start. Masukkan jumlah pemain: 4. Masukkan nama pemain 1: 'ghina'. Nama harus diawali huruf besar. Masukkan nama lain: 'Ghina'. Masukkan nama pemain 2: 'Sahla'. Masukkan nama pemain 3: 'Naya'. Masukkan nama pemain 4: 'Naya'. Nama sudah digunakan. Masukkan nama lain: 'Kate'. Urutan pemain: Ghina - Naya - Sahla - Kate. Setiap pemain mendapatkan 7 kartu acak. Kartu discard top: hijau-7 Giliran Ghina. yes</pre>

Pemain

Permainan dapat dimainkan oleh 2 hingga 4 pemain. Program akan meminta input jumlah pemain terlebih dahulu. Jika jumlah pemain berada di luar rentang tersebut, program akan meminta input ulang. Setelah jumlah pemain valid, setiap pemain diminta untuk memasukkan nama pemain.

Nama pemain harus unik sehingga jika terdapat nama yang sama dengan pemain lain, program akan meminta pemain untuk memasukkan nama lain.

Nama pemain harus diawali dengan huruf kapital. Nama pemain yang diinputkan juga harus berupa atom (bukan angka atau struktur term). Jika tidak memenuhi syarat tersebut, program akan meminta input ulang.

Kegunaan

Predikat `inputNamaPemain/3` secara rekursif mengumpulkan nama pemain satu per satu. Setiap nama divalidasi oleh `validasiNama` dengan tiga syarat: (1) input berupa atom, (2) huruf pertama kapital, (3) nama belum dipakai pemain lain.

Skenario dan Contoh Penggunaan

Skenario valid:

Contoh Program
<pre> ?- inputNamaPemain(2, [], Pemain). Masukkan nama pemain 1: 'Ghina'. Masukkan nama pemain 2: 'Ghani'. Pemain = ['Ghina','Ghani'] ?</pre>

Skenario tidak valid (nama tidak diawali kapital):

Contoh Program
<pre>Masukkan nama pemain 1: 'ghina'. Nama harus diawali huruf besar. Masukkan nama lain: </pre>

Skenario tidak valid (nama duplikat):

Contoh Program
<pre>Masukkan nama pemain 1: 'Ghina'. Masukkan nama pemain 2: 'Ghina'. Nama sudah digunakan. Masukkan nama lain: </pre>

Skenario tidak valid (input bukan atom):

Contoh Program
<pre>Masukkan nama pemain 1: 123. Input tidak valid. Masukkan nama lain: 'Ghina'. Masukkan nama pemain 2: 456. Input tidak valid. Masukkan nama lain: 'Sahla'.</pre>

Dampak terhadap state permainan

Predikat `inputNamaPemain` tidak mengubah database Prolog secara langsung. Hasilnya diteruskan ke `inisialisasiGame` lalu disimpan ke `urutanPemain/1`.

distribusiKartu

File `distribusiKartu.pl` berisi semua predikat yang bertugas: (1) membangun deck 108 kartu UNI secara lengkap (`buatDeckBaru`), (2) mengacak list (`acakList`), (3) membagi 7 kartu ke setiap pemain (`bagiKartuPemain`), dan (4) menetapkan kartu awal discard pile yang valid (`setKartuAwal`).

Kegunaan

Membangun, mengacak, dan mendistribusikan deck 108 kartu UNI ke setiap pemain, serta menentukan kartu awal yang valid untuk discard pile.

Skenario dan Contoh Penggunaan

Membuat deck baru:

Contoh Program
<pre> ?- buatDeckBaru(Deck), length(Deck, N). Deck = [kartu(biru,angka(1)),kartu(merah,skip),kartu(merah,angka(8)), kartu(kuning,angka(1)),kartu(biru,angka(6)),kartu(biru,reverse), kartu(hijau,drawTwo),kartu(biru,angka(3)),kartu(hijau,angka(2)), N = 112 ?</pre>

deck berisi 112 kartu baru (108 default dan 4 bonus mimic card)

Membagi kartu ke pemain (diinisialisasi otomatis oleh program):

Contoh Program
<pre> ?- kartuTangan('Ghina',X). X = [kartu(hijau,angka(1)),kartu(kuning,skip),kartu(kuning,skip), kartu(merah,drawTwo),kartu(merah,drawTwo),kartu(biru,angka(6)),...]</pre>

Kondisi tidak valid pada cekKartuValid:

Contoh Program
<pre> ?- cekKartuValid(kartu(hitam, wild)). no ?- cekKartuValid(kartu(merah, skip)). no ?- cekKartuValid(kartu(merah, angka(3))). yes</pre>

Dampak terhadap state permainan:

Predikat	Efek pada State
bagiKartuPemain	assertz(kartuTangan(P, Tangan)) untuk setiap pemain
buatDeckBaru	Hanya menghasilkan list, tidak mengubah database
acakList	Menggunakan random/3 untuk mengacak urutan
setKartuAwal	Dipakai inisialisasiGame, sisaDeck diisi dengan sisa deck

b. anggota

Perintah anggota digunakan untuk menampilkan daftar nama dan NIM seluruh anggota kelompok pengembang permainan UNI. Perintah ini dapat diakses kapan saja dari menu utama sebelum maupun sesudah permainan dimulai.

Contoh Program
<pre> ?- anggota. - 1. Ghina Emelia Yantes - 13525119 - 2. Sahla Nailah Salsabilla - 13525134 - 3. Cathrine Angel Siburian - 13525138 - 4. Nayla Putri Ghaisani - 13525140 yes</pre>

Kegunaan

Predikat `anggota/0` secara langsung mencetak empat baris terformat menggunakan `write/1` dan `nl`. Tidak ada argumen, tidak ada validasi, dan tidak mengakses database Prolog. Predikat ini murni bersifat informatif.

Dampak terhadap state permainan

Tidak mengubah database Prolog sama sekali. Tidak ada fakta yang di-assert maupun di-retract.

c. panduan

Perintah `panduan` menampilkan panduan dan regulasi lengkap cara bermain UNI, mencakup cara memulai game, alur bermain kartu, kartu spesial, mekanisme seruan "UNI", serta mengingat perintah `help`.

Contoh Program
<pre> ?- panduan. ===== PANDUAN DAN REGULASI PERMAINAN UNI ===== [1] CARA MEMULAI GAME ----- Ketik "start." di menu utama. Pilih mode: 1. Mode Klasik - 2 hingga 4 pemain, individu. 2. Mode Turnamen - tepat 4 pemain, sistem tim 2v2. Masukkan nama tiap pemain (diawali huruf kapital, unik). Setiap pemain otomatis mendapat 7 kartu acak. [2] ALUR GILIRAN ----- Pada giliranmu, tersedia aksi utama dan aksi pendukung. Ketik "lihatCommand." untuk melihat daftar aksi tersedia. (Tidak seluruh panduan ditampilkan demi kerapian tabel)</pre>

Kegunaan

Predikat `panduan/0` mencetak seluruh panduan permainan ke layar menggunakan serangkaian `write/1` dan `nl`. Tidak menerima argumen dan tidak mengakses database Prolog. Dapat dipanggil kapan saja — sebelum maupun sesudah permainan dimulai.

Dampak terhadap state permainan

Tidak mengubah database Prolog sama sekali.

d. help

Perintah `help` menampilkan daftar perintah yang tersedia selama sesi permainan berlangsung. Perintah ini sensitif terhadap konteks: jika permainan belum dimulai, `help` menampilkan pesan pengingat untuk mengetik `start`; jika permainan sudah berjalan, ditampilkan seluruh daftar perintah aktif.

Skenario permainan belum dimulai:

Contoh Program

```
| ?- help.  
Silakan ketik "start." untuk memulai permainan.  
  
yes
```

Skenario permainan aktif (mode klasik):

Contoh Program

```
| ?- help.  
  
--- DAFTAR PERINTAH AKTIF PERMAINAN ---  
+-----+  
| AKSI UTAMA (mengakhiri giliran):  
| mainkanKartu(N).      : Mainkan kartu ke-N.  
| uni(N).              : Mainkan kartu ke-N saat kartu tepat 2.  
| ambilKartu.          : Ambil kartu dari deck.  
| tantang.             : Tantang Wild Draw Four lawan.  
| sembunyikanKartu(N).  : Sembunyikan kartu ke-N.  
| tampilkanKartu.       : Tampilkan kembali kartu tersembunyi.  
| godsHand.            : Coba keberuntungan (peluang 15%).  
| AKSI PENDUKUNG (tidak mengakhiri giliran):  
| lihatKartu.          : Lihat kartu di tanganmu.  
| cekInfo.             : Lihat status meja & jumlah kartu.  
| lihatCommand.        : Tampilkan aksi yang tersedia.  
| tangkap>Nama).       : Tangkap pemain yang lupa UNI.  
| saveGame.            : Simpan permainan.  
| loadGame.            : Muat permainan tersimpan.  
+-----+  
Ketik "panduan." untuk penjelasan lengkap setiap perintah.  
  
(47 ms) yes
```

Skenario permainan aktif (mode turnamen):

Contoh Program

```
| ?- help.  
  
--- DAFTAR PERINTAH AKTIF PERMAINAN ---  
+-----+  
| AKSI UTAMA (mengakhiri giliran):  
| mainkanKartu(N).      : Mainkan kartu ke-N.  
| uni(N).              : Mainkan kartu ke-N saat kartu tepat 2.  
| ambilKartu.          : Ambil kartu dari deck.  
| tantang.             : Tantang Wild Draw Four lawan.  
| sembunyikanKartu(N).  : Sembunyikan kartu ke-N.  
| tampilkanKartu.       : Tampilkan kembali kartu tersembunyi.  
| godsHand.            : Coba keberuntungan (peluang 15%).  
| swapKartu(NoKu,NoTeman). : Tukar kartu dgn teman setim (Turnamen).  
| AKSI PENDUKUNG (tidak mengakhiri giliran):  
| lihatKartu.          : Lihat kartu di tanganmu.  
| cekInfo.             : Lihat status meja & jumlah kartu.  
| lihatCommand.        : Tampilkan aksi yang tersedia.  
| tangkap>Nama).       : Tangkap pemain yang lupa UNI.  
| saveGame.            : Simpan permainan.  
| loadGame.            : Muat permainan tersimpan.  
+-----+  
Ketik "panduan." untuk penjelasan lengkap setiap perintah.  
  
(46 ms) yes
```

Kegunaan

Predikat `help/0` menggunakan percabangan `->` untuk mendeteksi apakah fakta `giliranSekarang(_)` ada di database. Jika ada, permainan sedang berjalan dan daftar

perintah aktif dicetak. Jika tidak ada, ditampilkan pesan pengingat. Dalam mode turnamen, blok kondisional (`modeTurnamen -> ... ; true`) menambahkan baris `swapKartu(I, J)` di akhir daftar secara dinamis.

Dampak terhadap state permainan

Tidak mengubah database Prolog. Predikat `help/0` hanya membaca fakta `giliranSekarang/1` dan `modeTurnamen/0` tanpa memodifikasi apa pun.

e. `exit`

Perintah `exit` digunakan untuk keluar dari program UNI. Perintah ini bersifat konteks-sensitif: jika tidak ada permainan yang sedang berjalan, program langsung menutup sesi. Jika permainan sedang aktif, program menanyakan apakah pemain ingin menyimpan terlebih dahulu sebelum keluar. Jika pemain memilih untuk menyimpan, `saveGame` dieksekusi otomatis sebelum keluar.

Skenario tidak ada permainan yang berjalan:

Contoh Program
?- exit. Anda sudah keluar sebelumnya, tidak ada permainan yang sedang berjalan. yes

Skenario permainan berjalan, memilih simpan sebelum keluar:

Contoh Program
?- exit. Permainan sedang berjalan, simpan permainan saat ini sebelum keluar? (y/n): y. Melanjutkan ke proses penyimpanan game... Masukkan nama file penyimpanan: tes. Status permainan berhasil disimpan ke tes.txt. Terima kasih telah bermain UNI! Sampai jumpa lagi. true ? yes

Skenario permainan berjalan, memilih keluar tanpa menyimpan:

Contoh Program
?- exit. Permainan sedang berjalan, simpan permainan saat ini sebelum keluar? (y/n): n. Keluar tanpa menyimpan... Terima kasih telah bermain UNI! Sampai jumpa lagi. yes

Skenario input tidak valid saat ditanya simpan atau tidak:

Contoh Program
<pre> ?- exit. Permainan sedang berjalan, simpan permainan saat ini sebelum keluar? (y/n): gatau. Pilihan tidak valid! Masukkan "y." atau "n." Permainan sedang berjalan, simpan permainan saat ini sebelum keluar? (y/n): </pre>

Kegunaan

Predikat `exit/0` memiliki tiga klausa yang dieksekusi secara berurutan menggunakan `cut (!)`. Klausa pertama menangani kondisi tidak ada permainan (`\+ giliranSekarang(_)`). Klausa kedua menangani kondisi permainan aktif dengan meminta konfirmasi penyimpanan — jika jawaban `yes` atau `'y'`, maka `saveGame` dipanggil sebelum `exit_aksi`; jika `no` atau `'n'`, langsung ke `exit_aksi`; jika input tidak valid, pesan error ditampilkan dan `exit` dipanggil ulang secara rekursif. Klausa ketiga (`exit_aksi`) memanggil `hapusDataLama` untuk membersihkan seluruh fakta dinamis dari database Prolog.

Dampak terhadap state permainan

Memanggil `hapusDataLama/0` yang me-retractall seluruh fakta dinamis permainan: `urutanPemain/1`, `giliranSekarang/1`, `kartuTangan/2`, `kartuTeratas/1`, `warnaAktif/1`, `arahPermainan/1`, `sisadeck/1`, `statusAncaman/1`, `modeTurnamen/0`, `tim1/1`, `tim2/1`, dan seluruh fakta bonus lainnya. Setelah `exit` berhasil, database Prolog bersih seperti saat program pertama kali dijalankan.

f. mainkanKartu

Perintah `mainkanKartu/1` (berserta *helper*-nya) berisi predikat yang bertugas: (1) memvalidasi status ancaman dan aturan kecocokan kartu dengan arena (`cekValid`), (2) menghapus kartu yang dipilih dari daftar tangan pemain (`hapusKartuDariTangan/1`), (3) memperbarui *state* meja seperti kartu teratas, warna aktif, dan indeks tersembunyi (`updateWarnaAktif/1`, `updateIndeksTersembunyi/2`), serta (4) memicu efek dari kartu yang dimainkan dan mengevaluasi status akhir (`eksekusiEfek/0`, `endGame/0`).

Kegunaan

`mainkanKartu.pl` adalah file yang digunakan pemain untuk menaruh satu kartu dari tangannya ke tumpukan buangan (*discard pile*) berdasarkan indeks/nomor urut kartu. Perintah ini secara otomatis akan memvalidasi kecocokan kartu dengan arena, mengeksekusi efek dari kartu tersebut, serta memperbarui alur permainan (seperti giliran, status Uno, dan kondisi menang). Pemanggilannya dilakukan dengan cara mengetik `mainkanKartu(<NomorUrut>)`.

Skenario dan Contoh Penggunaan

Skenario valid

(Warna atau angka/jenis cocok):

Contoh Program
<pre> ?- mainkanKartu(3). Ket memainkan kartu: biru-6. Tidak ada efek spesial. Giliran Naya.</pre>

Memainkan kartu aksi (Skip/reverse/draw two):

Contoh Program
<pre> ?- mainkanKartu(1). Ket memainkan kartu: biru-skip. Pemain berikutnya telah dilewati Giliran Ket.</pre>

Memainkan kartu hitam (Wild/mimic):

Contoh Program
<pre> ?- mainkanKartu(2). Naya memainkan kartu: hitam-wild. Pilih warna: (merah, kuning, hijau, biru). merah. Warna berubah menjadi merah. Giliran Ket.</pre>

Kartu terakhir (Menang):

Contoh Program
<pre> ?- mainkanKartu(1). Permainan selesai! Naya menghabiskan semua kartunya! Berikut perhitungan poin sisa kartu. Ket: merah-2 + biru-reverse + biru-7 + merah-reverse + merah-5 + biru-2 = 2 + 10 + 7 + 10 + 5 + 2 = 36 poin Naya: kartu habis = 0 poin Urutan pemenang: 1. Naya (0 poin) 2. Ket (36 poin) Selamat, Naya menjadi pemenang!</pre>

Skenario tidak valid

Kartu tidak cocok dengan meja/nomor urut (indeks) salah:

Contoh Program
<pre> ?- mainkanKartu(3). Gagal memainkan. Nomor urut salah atau kartu tidak cocok dengan meja!</pre>

Sedang terkena ancaman wild draw four:

Contoh Program
?- mainkanKartu(1). Anda sedang terkena efek Wild Draw Four! Pilih perintah: "tantang." atau "ambilKartu."

Dampak terhadap state permainan

Berikut adalah penjelasan deskriptif mengenai dampak perintah (saat pemain melempar kartu) terhadap *state* permainan:

- Pembaruan Kartu di Tangan dan di Meja Saat perintah dijalankan, *state* *kartuTangan/2* dan *kartuTersembunyi/2* akan langsung diperbarui. Hal ini terjadi karena kartu ditarik dari tangan pemain, sehingga sisa indeks kartu yang ada harus digeser untuk menyesuaikan urutan. Di saat yang sama, *state* *kartuTeratas/1* dan *warnaAktif/1* akan ditimpa sepenuhnya dengan identitas kartu yang baru saja dilempar ke tumpukan meja.
- Pembaruan Status Seruan dan Ancaman Perintah ini akan memengaruhi status aktif pemain. *State* *statusUni/1* akan dihapus secara paksa (menggunakan *retractall*) untuk membatalkan status seruan "UNO" dari pemain tersebut. Selanjutnya, *state* *statusAncaman/1* akan dievaluasi dan diperbarui: statusnya menjadi aktif jika kartu yang dimainkan adalah *Wild Draw Four*, atau menjadi aman jika yang dimainkan adalah kartu jenis lain.
- Pencatatan Riwayat Permainan Sistem akan menyimpan catatan kondisi meja sesaat sebelum efek kartu baru dieksekusi. Informasi ini direkam ke dalam *state* riwayat permainan, yaitu *warnaSebelumnya/1*, *jenisSebelumnya/1*, dan *pemainSebelumnya/1*.
- Modifikasi Alur Permainan (Dampak Tidak Langsung) Secara tidak langsung, perintah ini akan memicu pemanggilan predikat *eksekusiEfek*. Predikat inilah yang pada akhirnya akan memodifikasi *state* *giliranSekarang/1* (menentukan siapa yang maju berikutnya) atau *arahPermainan/1* (membalikkan arah putaran), sesuai dengan efek khusus dari kartu yang baru saja dimainkan.

g. ambilKartu

Perintah *ambilKartu* (berserta *helper*-nya *ambilN*) berisi predikat yang bertugas mengecek ketersediaan kartu pada tumpukan (*deck*), menarik sejumlah kartu dari atas *deck* (1 kartu untuk giliran biasa, atau sejumlah *N* kartu untuk efek penalti), menggabungkan (*append*) kartu yang ditarik tersebut ke dalam daftar tangan pemain (*kartuTangan/2*), memperbarui memori sisa *deck*, serta memicu perpindahan giliran otomatis ke pemain selanjutnya (*gantiGiliran/0*).

Kegunaan

Perintah ini terbagi menjadi dua kegunaan utama:

1. *ambilKartu*. (tanpa argumen): Digunakan secara manual oleh pemain pada gilirannya ketika mereka tidak memiliki kartu yang sah untuk dimainkan, atau

memilih untuk tidak mengeluarkan kartu. Pemain akan mendapat 1 kartu acak dan gilirannya langsung berakhir.

2. `ambilKartu(<Pemain>, <Jumlah>)`:. Merupakan fungsi internal program untuk mengeksekusi hukuman penalti. Jika kartu sebelumnya adalah *Draw Two* atau *Wild Draw Four*, sistem akan memanggil perintah ini untuk memaksa pemain target mengambil sejumlah kartu sesuai efek hukuman tersebut.

Skenario dan Contoh Penggunaan.

Skenario valid Pengambilan Kartu Normal:

Contoh Program
<pre> ?- ambilKartu. Anda mengambil kartu: hijau-2 Giliran Ket.</pre>

Eksekusi Efek Hukuman:

Contoh Program
<pre> ?- ambilKartu. Anda memilih tidak menantang. A mengambil 4 kartu dari deck. Giliran K.</pre>

Skenario tidak valid Gagal Eksekusi:

Contoh Program
<pre> ?- ambilKartu. Gagal memainkan. Deck kosong!</pre>

Dampak terhadap state permainan

Memperbarui `kartuTangan`, `kartuTeratas`, `warnaSebelumnya`, `jenisSebelumnya`, `pemainSebelumnya`, dan `statusAncaman` menggunakan `retract` dan `asserta`. Menghapus `statusUni` secara paksa dengan `retractall`.

h. `cekInfo`

Perintah `cekInfo` menampilkan keadaan permainan saat ini. Pada mode `klasik`, informasi yang akan ditampilkan adalah kartu teratas, urutan pemain, dan seluruh nama pemain beserta jumlah kartu yang dimiliki. Pada mode `turnamen`, informasi yang akan ditampilkan adalah kartu teratas, anggota kedua tim, urutan pemain, dan seluruh nama pemain beserta jumlah kartu yang dimiliki.

Kegunaan

Rules `cekInfo` akan memeriksa apakah permainan dalam mode `klasik` atau `turnamen`. Jika permainan dalam mode `turnamen`, `cekInfo` akan menampilkan masing-masing anggota tim. `cekInfo.pl` juga memiliki rules `hitungKartuTersembunyi` untuk menghitung banyak kartu yang tersembunyi menggunakan rules `hitungIndeksTersembunyi` secara rekursif, nantinya jumlah kartu pemain yang

asli dikurangi dengan jumlah kartu yang disembunyikan sebelum ditampilkan. Untuk menampilkan seluruh nama pemain dan jumlah kartu yang dimiliki, cekInfo.pl menggunakan rules tampilkanDetailPemain secara rekursif hingga semua pemain berhasil ditampilkan.

Skenario dan Contoh Penggunaan

Skenario valid (permainan mode klasik):

Contoh Program
<pre> ?- cekInfo. Kartu discard top: hijau-drawTwo. Urutan pemain: Sahla - Bimo - Wishnu. Nama pemain 1: Sahla Jumlah kartu : 6 Nama pemain 2: Bimo Jumlah kartu : 6 Nama pemain 3: Wishnu Jumlah kartu : 4 yes</pre>

Skenario valid (permainan mode turnamen):

Contoh Program
<pre> ?- cekInfo. Kartu discard top: hijau-7. Tim 1 : S, T Tim 2 : U, V Urutan pemain: S - U - T - V. Nama pemain 1: S Jumlah kartu : 7 Nama pemain 2: U Jumlah kartu : 7 Nama pemain 3: T Jumlah kartu : 7 Nama pemain 4: V Jumlah kartu : 7 yes</pre>

Skenario tidak valid (permainan belum dimulai):

Contoh Program
?- cekInfo. Tidak ada permainan yang sedang berjalan. Ketik "start." untuk mulai. yes

Dampak terhadap state permainan:

cekInfo tidak mempengaruhi jalannya permainan, tetapi membantu pemain untuk melihat keadaan permainan saat ini.

- i. **tantang**
Perintah `tantang` hanya bisa digunakan jika status ancaman pemain aktif (terkena efek kartu Wild Draw Four dari pemain sebelumnya). Selain perintah `tantang`, pemain juga bisa menggunakan perintah `ambilKartu` jika pemain sebelumnya memainkan kartu Wild Draw Four.

Kegunaan

Jika pemain sebelumnya memainkan kartu Wild Draw Four padahal masih memiliki kartu yang cocok dengan warna atau angka aktif, maka tantangan berhasil dan pemain sebelumnya mendapat hukuman yaitu `ambilKartu` sebanyak 4 kali. Jika tantangan gagal, maka penantang mendapat hukuman yaitu `ambilKartu` sebanyak 6 kali.

Skenario dan Contoh Penggunaan

Skenario valid (status ancaman aktif, tantang berhasil):

Contoh Program
?- tantang. Tantangan dilakukan! Memeriksa kartu Sahla... Tantangan berhasil! Sahla secara ilegal memainkan Wild Draw Four dan mendapatkan 4 kartu acak. Giliran Wishnu.

Skenario valid (status ancaman aktif, tantang gagal):

Contoh Program
?- tantang. Tantangan dilakukan! Memeriksa kartu Bimo... Tantangan gagal. Sahla mendapatkan 6 kartu acak. Giliran Wishnu.

Skenario tidak valid (pemain sebelumnya tidak memainkan kartu Wild Draw Four):

Contoh Program
?- tantang. Tidak ada kartu Wild Draw Four yang bisa ditantang saat ini. yes

Skenario tidak valid (permainan belum dimulai):

Contoh Program
?- tantang. Tidak ada permainan yang sedang berjalan. Ketik "start." untuk mulai. yes

Dampak terhadap state permainan

Kondisi	Perubahan State	Keterangan
ambilKartu(Ter tantang, 4)	Diperbarui	Kartu pemain sebelumnya bertambah 4 jika tantang berhasil.
ambilKartu(Pen antang, 6)	Diperbarui	Kartu pemain saat ini bertambah 6 jika tantang gagal.

- j. uni
- Perintah uni (beserta *helper*-nya ambilKartu/3) berisi predikat yang bertugas: (1) mengevaluasi jumlah kartu pemain (wajib berjumlah 2 sebelum dimainkan), (2) mengambil kartu sesuai nomor urut yang dipilih, (3) mengeksekusi pelemparan kartu sekaligus melindungi pemain dari hukuman dengan mendeklarasikan status aman (tandaAman), serta (4) memberikan sanksi penalti kartu jika perintah dipanggil pada kondisi jumlah kartu yang menyalahi aturan.

Kegunaan

ini merupakan perintah wajib pengganti mainkanKartu ketika seorang pemain memegang tepat 2 kartu di tangannya dan ingin membuang 1 kartu ke meja. Dengan menggunakan perintah ini, pemain secara bersamaan membuang kartu dan menyerukan "UNI!" (tersisa 1 kartu). Hal ini akan menandai pemain tersebut dalam status "aman", sehingga tidak bisa dihukum oleh pemain lain dengan perintah tangkap.

Skenario dan Contoh Penggunaan.

Skenario valid

Contoh Program
?- uni(1). Naya memainkan kartu: merah-2. Naya menyerukan UNI! yes

Skenario tidak valid

Jumlah kartu bukan 2 - kena penalti.

Contoh Program
<pre> ?- uni(2). Perintah tidak valid! Kartu tidak membuat sisa tangan menjadi 1. Naya mendapatkan 1 kartu penalti acak. yes</pre>

Skenario tidak valid

Kartu tidak cocok/indeks salah:

Contoh Program
<pre> ?- uni(1). Gagal! Kartu yang dipilih tidak valid, tidak cocok dengan kartu di meja, atau indeks di luar batas. yes</pre>

Dampak terhadap state permainan

- Jika berhasil: Memperbarui kartuTangan dan kartuTeratas (menggunakan retract dan assertz), memicu state tandaiAman, serta memicu perubahan warnaAktif dan giliranSekarang.
- Jika pelanggaran jumlah kartu: Memperbarui kartuTangan (ditambah kartu penalti) dan sisaDeck melalui predikat hukumAmbilKartu, serta memperbarui giliranSekarang.

k. tangkap

Kegunaan

Perintah ini digunakan sebagai mekanisme pengawasan antar pemain. Pada giliran aktifnya, seorang pemain dapat memanggil perintah ini untuk menghukum pemain lain (Target) yang dicurigai lupa menyerukan "UNI" ketika kartunya tersisa tepat 1 buah. Perintah ini memiliki risiko dua arah, jika tebakan benar, Target dihukum, namun jika salah menuduh, penangkaplah yang akan mendapat hukuman.

Skenario dan Contoh Penggunaan.

Skenario valid tangkapan berhasil:

Contoh Program
<pre> ?- tangkap('Ket'). Ket tertangkap tidak menyerukan UNI. Ket mendapatkan 2 kartu penalti. yes</pre>

Skenario tidak valid

Skenario Tuduhan Salah - Penangkap Kena Penalti:

Contoh Program
<pre> ?- tangkap('Ket'). Tuduhan salah! Target tidak melanggar aturan. Ket mendapatkan 1 kartu penalti secara acak. yes</pre>

Dampak perintah terhadap jalannya permainan (state permainan)

- Tangkapan valid: mengubah kartuTangan target (ditambah 2 kartu penalti) dan sisaDeck (berkurang) secara tidak langsung melalui hukumAmbilKartu. Memperbarui giliranSekarang melalui gantiGiliran.
- Tuduhan salah: mengubah kartuTangan penangkap (ditambah 1 kartu penalti) dan sisaDeck (berkurang). Memperbarui giliranSekarang.

1. lihatKartu

Perintah `lihatKartu` adalah perintah pendukung yang berfungsi untuk menampilkan seluruh kartu yang sedang dipegang oleh pemain yang aktif pada gilirannya. Perintah ini memanfaatkan konsep rekursi untuk menelusuri daftar (*list*) kartu milik pemain tersebut dan mencetaknya ke layar satu per satu lengkap dengan nomor urutnya.

Kegunaan :

1. Pemain dapat melihat kartu apa saja yang mereka miliki untuk menyusun strategi sebelum mengambil aksi utama.
2. Menampilkan status khusus kartu yang sedang disembunyikan (fitur bonus), sehingga pemain yang bersangkutan tetap bisa melihat kartu miliknya yang sedang tidak terlihat oleh pemain lain.
3. Dalam mode turnamen, pemain bisa melihat kartu milik teman se-tim nya.

Contoh Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. biru-skip 2. hijau-reverse 3. biru-6 4. merah-reverse 5. biru-2 6. hijau-1 7. kuning-0 yes</pre>

Saat berhasil menyembunyikan kartu, perintah `lihatKartu` akan menandai kartu yang telah disembunyikan.

Contoh Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. biru-skip 2. hijau-reverse 3. biru-6 4. merah-reverse (Kartu tersembunyi) 5. biru-2 6. hijau-1 7. kuning-0 (31 ms) yes</pre>

Perintah `lihatKartu` saat mode turnamen akan menampilkan kartu yang dimiliki oleh pemain dan timnya.

Contoh Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. merah-drawTwo 2. hijau-4 3. kuning-6 4. biru-3 5. merah-skip 6. kuning-reverse 7. merah-drawTwo Berikut kartu yang teman satu tim anda miliki (Ket). 1. hijau-4 2. merah-6 3. hitam-wildDrawFour 4. hitam-wild 5. biru-2 6. kuning-9 7. hijau-7 yes</pre>

m. `lihatCommand`

Perintah `lihatCommand` berfungsi sebagai antarmuka pemandu bagi pemain yang sedang mendapatkan giliran. Perintah ini akan mencetak daftar aksi utama dan aksi pendukung yang dapat dieksekusi oleh pemain pada kondisi state permainan saat itu. Perintah ini bekerja secara deklaratif dengan membaca kondisi terkini dari permainan untuk menampilkan daftar perintah (Aksi Utama dan Aksi Pendukung) yang valid dan dapat dieksekusi oleh pemain tersebut pada baris perintah.

Kegunaan :

1. Membantu pemain (terutama pemain pemula) untuk mengetahui opsi apa saja yang bisa mereka lakukan saat giliran mereka sedang berlangsung.
2. Memastikan transparansi aturan permainan, dimana pemain hanya diperlihatkan aksi yang memang legal secara aturan berdasarkan *state* permainan saat itu.

Dalam permainan UNI terdapat dua mode permainan yaitu mode klasik dan mode turnamen. Saat mode klasik, command yang ditampilkan adalah aksi-aksi yang legal dalam mode ini.

Contoh Program
<pre> ?- lihatCommand. Aksi utama yang tersedia: 1. mainkanKartu(NomorUrut) 2. ambilKartu 3. uni(NomorUrut) 4. sembunyikanKartu(NomorUrut) 5. godsHand. Aksi pendukung yang tersedia: 1. lihatCommand 2. lihatKartu 3. cekInfo (32 ms) yes</pre>

Dalam mode turnamen terdapat aksi tambahan yang terlihat dalam command yaitu `swapKartu` (fitur tambahan dalam mode turnamen).

Contoh Program
<pre> ?- lihatCommand. Aksi utama yang tersedia: 1. mainkanKartu(NomorUrut) 2. ambilKartu 3. uni(NomorUrut) 4. sembunyikanKartu(NomorUrut) 5. godsHand. 6. swapKartu(NomorUrutKartuKu, NomorUrutKartuTeman) Aksi pendukung yang tersedia: 1. lihatCommand 2. lihatKartu 3. cekInfo (15 ms) yes</pre>

- n. **endGame**
 Perintah `endGame` adalah fungsi kontrol utama yang dieksekusi secara otomatis ketika kondisi akhir permainan terpenuhi, yaitu saat salah satu pemain berhasil menghabiskan seluruh kartu di tangannya (`kartuTangan(Pemain, [])`). Perintah ini menghentikan *gameplay loop*, mengalkulasi bobot nilai dari sisa kartu yang dipegang oleh seluruh pemain, melakukan pengurutan peringkat secara deklaratif, dan menampilkan papan peringkat akhir (*leaderboard*) pada terminal.

Output endGame pada mode klasik.

Contoh Program
<pre>Permainan selesai! Ket menghabiskan semua kartunya! Berikut perhitungan poin sisa kartu. Ket: kartu habis = 0 poin Sah: merah-9 + merah-skip + kuning-6 + merah-1 + biru-7 + merah-skip + merah-4 = 9 + 10 + 6 + 1 + 7 + 10 + 4 = 47 poin Nay: merah-8 + merah-0 + biru-9 + biru-1 + hitam-wild + kuning-skip + hitam-wildDrawFour + hitam-wildDrawFour + biru-1 = 8 + 1 + 8 + 1 + 20 = ... Urutan pemenang: 1. Ket (0 poin) 2. Sah (47 poin) 3. Nay (89 poin) Selamat, Ket menjadi pemenang!</pre>

Output endGame pada mode turnamen..

Contoh Program
<pre>Permainan selesai! Ket menghabiskan semua kartunya! Berikut perhitungan poin sisa kartu. Ket: kartu habis = 0 poin Gin: merah-drawTwo + merah-6 + merah-2 + kuning-reverse + biru-4 + hijau-9 = 10 + 6 + 2 + 10 + 4 + 9 = 41 poin Nay: merah-6 + merah-2 + kuning-drawTwo = 6 + 2 + 10 = 18 poin Sah: hijau-6 + merah-9 + kuning-drawTwo + biru-skip = 6 + 9 + 10 + 10 = 35 poin Berikut perhitungan poin untuk masing-masing tim. Tim 1 (Ket, Nay) : 0 + 6 = 6 poin Tim 2 (Gin, Sah) : 87 + 35 = 122 poin Selamat, Tim 1 menjadi pemenang! yes</pre>

Kegunaan

1. Mentransformasikan sisa kartu fisik menjadi representasi nilai numerik/integer secara akurat berdasarkan aturan permainan dimana kartu angka bernilai sesuai angkanya, kartu aksi bernilai 10, dan kartu *Wild/mimic* bernilai 20.
2. Mengurutkan posisi akhir pemain dari yang terbaik hingga terburuk dengan tiga lapis validasi presisi: total poin terkecil, jumlah sisa kartu fisik tersedikit, dan urutan giliran bermain paling awal.
3. Menyajikan visualisasi *breakdown* perhitungan matematika skor kepada pemain sebelum menobatkan pemenang utama secara resmi.

Dampak terhadap state permainan

Predikat	Jenis	Efek Pada State
endGame/0 & endGameKlasik/0	Utama (Kontrol)	Mengarahkan alur akhir game dan mengeksekusi sekuens penutup: menampilkan pemenang, memicu kalkulasi, dan mencetak <i>leaderboard</i> .
hitungNilaiKartu/2	Evaluasi (Fakta)	Memetakan struktur kartu ke nilai poinnya menggunakan <i>pattern matching</i> dan operator <i>cut</i> (!) untuk efisiensi.
hitungTotalPoin/2	Evaluasi (Rekursif)	Menjumlahkan seluruh nilai kartu di tangan pemain menggunakan konsep rekursi list (<i>Head-Tail</i>).
cariIndeks/3	Utilitas	Mencari posisi indeks (berbasis 1) seorang pemain dalam list urutanPemain untuk keperluan kriteria <i>tie-breaker</i> .
dataPemain/2	Struktur Data	Mengonstruksi data profil pemain menjadi format data (Player, Poin, Urutan)
comparePeringkat/3	Logika Komparasi	Menentukan prioritas peringkat (< atau >) antara dua pemain berdasarkan total poin, dan menggunakan urutan main jika poin seri.
sortManual/2 & insertSorted/3	Algoritma	Mengimplementasikan algoritma <i>Insertion Sort</i> murni berbasis list di Prolog untuk menyusun data pemain dari peringkat terbaik ke terburuk.
cetakNamaKartu/1 & cetakNilaiKartu/1	I/O (Rekursif)	Mencetak rincian kartu dan nilainya yang dipisahkan dengan string ' + '
pemainMenang/1	Validasi	Mengecek kondisi basis kemenangan, yaitu fakta kartuTangan (Pemain, []) yang bernilai list kosong.

o. saveGame

Perintah `saveGame` adalah rules yang digunakan untuk menyimpan kondisi permainan saat ini ke dalam sebuah file `.txt`. Jika dipanggil, pemain akan diminta untuk memasukkan nama file. Jika file sudah ada, isi file akan ditimpa dengan kondisi permainan terbaru. Jika file belum ada, file akan otomatis dibuat. Informasi permainan yang ditulis kedalam file yaitu:

1. mode permainan (klasik/turnamen)
2. anggota tim (jika mode turnamen)

3. urutan pemain
4. giliran
5. discard top (kartu teratas)
6. warna aktif
7. arah permainan
8. pemain yang sudah mencapai status UNI
9. status ancaman (aman = pemain tidak terkena efek kartu Wild Draw Four)
10. warna kartu yang dimainkan pemain sebelumnya
11. jenis kartu yang dimainkan pemain sebelumnya
12. pemain sebelumnya
13. seluruh kartu yang dimiliki tiap pemain
14. seluruh kartu tersembunyi tiap pemain
15. kartu aksi terakhir yang dimainkan
16. sisa deck (sisa kartu yang ada di meja)

Kegunaan

Rules `saveGame` hanya bisa digunakan jika `startGame` sudahRules `saveGame` akan memeriksa apakah permainan dalam mode klasik atau turnamen. Jika permainan dalam mode turnamen, `saveGame` akan menyimpan masing-masing anggota tim ke dalam file. `saveGame` menggunakan rules `simpanKartuPemain` secara rekursif untuk menyimpan seluruh kartu masing-masing pemain ke dalam file dan rules `simpanKartuTersembunyi` untuk menyimpan seluruh kartu tersembunyi masing-masing pemain ke dalam file.

Skenario dan Contoh Penggunaan

Skenario valid (permainan sudah dimulai):

Contoh Program
<pre> ?- saveGame. Masukkan nama file penyimpanan: lancar. Status permainan berhasil disimpan ke lancar.txt.</pre>
Contoh Isi File
<pre>urutan_pemain:['Sahla','Bimo','Wishnu']. giliran:'Bimo'. discard_top:hijau-drawTwo. warna_aktif:hijau. arah_permainan:kanan. status_UNI:[]. status_ancaman:aman. warna_sebelumnya:hijau. jenis_sebelumnya:drawTwo. pemain_sebelumnya:'Sahla'. kartu('Sahla'):[hijau-reverse,biru-angka(4),hijau-angka(7),hitam-wild,...]. kartu('Bimo'):[biru-reverse,hijau-angka(9),kuning-angka(6),...]. kartu('Wishnu'):[hijau-angka(1),hitam-mimic,hijau-angka(6),...]. tersembunyi('Sahla'):[]. tersembunyi('Bimo'):[1]. tersembunyi('Wishnu'):[2].</pre>

```
kartu_aksi_terakhir:hijau-drawTwo-'Sahla'-12.  
sisa_deck:[biru-angka(7),kuning-angka(2),merah-skip,hijau-angka(8),...  
].
```

Skenario tidak valid (permainan belum dimulai):

Contoh Program
?- saveGame. Tidak ada permainan yang sedang berjalan. yes

Skenario tidak valid (status ancaman pemain tidak aman):

Contoh Program
?- saveGame. Tidak dapat menyimpan saat terkena efek Wild Draw Four. Pilih perintah: "tantang." atau "ambilKartu." terlebih dahulu. yes

Dampak terhadap state permainan:

saveGame tidak mempengaruhi jalannya permainan, tetapi membantu pemain untuk menyimpan permainan saat ini supaya bisa dimainkan kembali di lain waktu.

p. loadGame

Kegunaan

loadGame berisi predikat yang bertugas meminta input nama file dari pemain dan memvalidasi keberadaannya di dalam direktori, membersihkan seluruh state permainan yang sedang berjalan saat ini (bersihkanStateLama), membaca isi file save baris per baris lalu memuat ulang fakta-fakta tersebut ke dalam memori permainan (bacaIsiFile), serta memanipulasi penggabungan teks secara manual untuk mendeteksi format .txt tanpa mengandalkan fungsi bawaan (built-in).

Skenario dan Contoh Penggunaan.

Skenario valid Permainan sedang berjalan disimpan dan File ditemukan:

Contoh Program
?- loadGame. Permainan sedang berjalan, simpan permainan saat ini? (y/n): y. Masukkan nama file penyimpanan: test. Status permainan berhasil disimpan ke test.txt. Melanjutkan ke proses pemuatan game.. Masukkan nama file yang akan dimuat: tes. Status permainan berhasil dimuat dari tes.txt. Melanjutkan giliran Sahla.

Skenario valid Permainan sedang berjalan tidak disimpan dan File ditemukan:

Contoh Program
<pre> ?- loadGame. Permainan sedang berjalan, simpan permainan saat ini? (y/n): n. Melanjutkan tanpa menyimpan... Masukkan nama file yang akan dimuat: tes. Status permainan berhasil dimuat dari tes.txt. Melanjutkan giliran Sahla.</pre>

Skenario tidak valid Permainan sedang berjalan disimpan dan File tidak ditemukan:

Contoh Program
<pre> ?- loadGame. Permainan sedang berjalan, simpan permainan saat ini? (y/n): y. Masukkan nama file penyimpanan: test. Status permainan berhasil disimpan ke test.txt. Melanjutkan ke proses pemuatan game... Masukkan nama file yang akan dimuat: sahla. Gagal! File sahla.txt tidak ditemukan di direktori.</pre>

Skenario tidak valid Permainan sedang berjalan tidak disimpan dan File tidak ditemukan:

Contoh Program
<pre> ?- loadGame. Permainan sedang berjalan, simpan permainan saat ini? (y/n): Melanjutkan tanpa menyimpan... Masukkan nama file yang akan dimuat: test. Gagal! File test.txt tidak ditemukan di direktori.</pre>

Dampak perintah terhadap jalannya permainan (state permainan)

Kondisi	Perubahan State	Keterangan
loadGame/ prosesLoadData	Memicu hapusDataLama; menambahkan assertz(sisaDeck([])) jika file usang dan deck kosong	Mengelola file save. Jika valid, memicu pengisian ulang data untuk dimuat ke dalam permainan.
hapusDataLama	Menggunakan retractall pada state yang sedang aktif	Menghapus bersih seluruh state permainan di memori dinamis agar siap ditimpa data baru dari file save.
tafsirData	Menggunakan assertz secara masif	berfungsi untuk membangun ulang seluruh database memori permainan.

Kondisi	Perubahan State	Keterangan
loadGame/ prosesLoadData	Memicu hapusDataLama; menambahkan assertz(sisaDeck([])) jika file usang dan deck kosong	Mengelola file save. Jika valid, memicu pengisian ulang data untuk dimuat ke dalam permainan.
assertIndeksTersembunyi	Menyuntikkan assertz(kartuTersembunyi(Pemain, Indeks))	Memulihkan urutan rahasia kartu pemain ke dalam state permainan.
konversiListKartu	Tidak ada perubahan pada state	Menerjemahkan (parsing) format string/atom dari file .txt menjadi struktur term kartu(Warna, Jenis) yang dikenali program.

q. godsHand

Perintah bonus. Ketika seorang pemain mengetik godsHand. di gilirannya, sistem melempar "dadu probabilitas" dengan peluang 15% sukses. Jika berhasil, satu kartu acak dari seorang pemain acak akan dipindahkan ke tangan pemain acak lainnya. Pemain hanya dapat menggunakan perintah ini sekali per giliran.

Kegunaan

Menambahkan elemen kejutan atau kejadian acak yang dapat membalikkan permainan. Dilindungi flag godsHandDipakai/1 agar tidak dipakai berulang kali dalam satu giliran.

Skenario dan Contoh Penggunaan

Skenario sukses (roll < 15):

Contoh Program
<pre> ?- godsHand. Tuhan telah berkehendak. Kartu biru-9 milik Kate berpindah ke tangan Ghina! Giliran Kate. true ? yes</pre>

Skenario gagal (roll >= 15):

Contoh Program
<pre> ?- godsHand. Tidak ada kejadian spesial. yes</pre>

Dipakai dua kali dalam satu giliran:

Contoh Program
<pre> ?- godsHand. Kamu sudah mencoba God's Hand di giliran ini! Silakan mainkanKartu(NomorUrut). atau ambilKartu. yes</pre>

Jika semua pemain hanya punya satu kartu

`semuaPemainSatuKartu` bernilai `true` → `godsHandEksekusi` tidak memindahkan kartu, giliran tetap berlanjut.

Dampak terhadap state permainan

Kondisi	Perubahan State	Keterangan
Roll sukses	<code>kartuTangan</code> diperbarui; <code>godsHandDipakai</code> ditambah; <code>gantiGiliran</code> dipanggil	Kartu berpindah dari sumber ke tujuan; giliran berganti
Roll gagal	<code>godsHandDipakai</code> ditambah	Tidak ada perpindahan kartu; giliran TIDAK berganti
Sudah dipakai	Tidak ada perubahan	Pesan peringatan ditampilkan

r. `sembunyikanKartu`

Perintah `sembunyikanKartu(NomorUrut)` adalah salah satu Aksi Utama (*Main Action*) dalam permainan UNI yang memungkinkan pemain aktif untuk menyembunyikan salah satu kartu di tangannya berdasarkan nomor urut kartu tersebut. Kartu yang disembunyikan akan diberi penanda khusus agar saat perintah `lihatKartu` dipanggil, kartu tersebut memunculkan teks (disembunyikan) dan efeknya tidak dapat dilihat oleh pemain lain sesuai dengan mekanisme bonus permainan.

Kegunaan

1. Membantu pemain mengelabui lawan dengan menyembunyikan kartu kunci (misalnya kartu *Wild* atau kartu angka tinggi) sehingga lawan tidak bisa memprediksi sisa kartu atau strategi yang akan dikeluarkan.
2. Mengubah properti atau representasi struktur internal dari suatu objek kartu di dalam tangan pemain dari kondisi normal menjadi sembunyi/tersembunyi.

Contoh Program
<pre> ?- sembunyikanKartu(4). Kartu merah-reverse berhasil disembunyikan. Giliran Ket. true ? yes</pre>

Saat kartu tersisa satu, pemain tidak dapat melakukan aksi sembunyikanKartu.

Contoh Program
<pre> ?- sembunyikanKartu(1).</pre> Tidak bisa menyembunyikan kartu jika hanya tersisa 1 kartu. yes

Jika kartu sudah dalam kondisi tersembunyi, maka kartu tersebut tidak dapat disembunyikan lagi.

Contoh Program
<pre>(16 ms) yes ?- sembunyikanKartu(1).</pre> Kartu tersebut sudah dalam status tersembunyi. true ? yes

Dampak terhadap state dalam permainan

- Kartu yang disembunyikan langsung berstatus "gaib" bagi pemain lain, mengubah psikologis permainan karena lawan harus menebak-nebak sisa kartu di tangan pemain tersebut.
- Alur permainan dipaksa bergerak maju ke pemain berikutnya melalui pemanggilan gantiGiliran. Pemain yang baru saja menyembunyikan kartu tidak bisa melakukan aksi utama lainnya (seperti mainkanKartu) sampai gilirannya berputar kembali.

Kondisi	Perubahan State	Keterangan
sembunyikanKartu(N omorUrut)	Menyuntikkan asserta(kartuTers embunyi(Pemain , NomorUrut)) jika validasi terpenuhi; Memperbarui <i>state</i> melalui predikat gantiGiliran.	Memeriksa jumlah kartu tangan dan batas indeks. Jika valid, menyembunyikan kartu terpilih dengan menambahkan <i>flag</i> rahasia, lalu otomatis mengalihkan giliran ke pemain berikutnya.
tampilkanKartu	Menggunakan retractall(kar	Menghapus bersih seluruh

	tuTersembunyi (Pemain, _)) pada aturan permainan yang aktif.	status/penanda kartu tersembunyi milik pemain aktif secara masif, sehingga status seluruh kartu tangan kembali normal (terbuka).
kartuTersembunyi_check	Tidak ada perubahan pada state.	Berfungsi sebagai wrapper atau pengecekan pasif untuk mendeteksi apakah indeks kartu tertentu milik pemain sedang dalam kondisi tersembunyi.

s. Mimic Card

Predikat efekMimic(mimic) bertugas menelusuri memori permainan untuk mencari data kartu aksi terakhir yang pernah dimainkan (kartuAksiTerakhir), menghitung selisih giliran sejak kartu aksi itu dikeluarkan, menyalin (copy/mimic) efek dari kartu aksi tersebut secara dinamis, serta menyediakan cadangan agar kartu ini bertindak sebagai kartu Wild biasa jika belum ada riwayat kartu aksi sama sekali di arena.

Kegunaan

Predikat ini dipicu secara otomatis oleh sistem (melalui eksekusiEfek(mimic)) ketika seorang pemain memainkan kartu spesial hitam-mimic. Kegunaannya adalah menciptakan kembali efek kartu (*Skip, Reverse, Draw Two, Wild Draw Four*) yang sudah lewat, sekaligus memungkinkan penggunaanya mengubah warna arena saat ini.

Skenario dan Contoh Penggunaan

Skenario valid

Menyalin Kartu Aksi:

Contoh Program
<pre> ?- mainkanKartu(5). Ket memainkan kartu: hitam-mimic. Menelusuri riwayat permainan... Kartu aksi terakhir yang dimainkan: merah-skip (oleh Ket, 6 giliran lalu) Kartu mimic menyalin efek skip! Pilih warna (merah, kuning, hijau, biru): hijau. Warna aktif sekarang: hijau. Pemain berikutnya telah dilewati Giliran Ket.</pre>

Belum Ada Riwayat:

Contoh Program
<pre> ?- mainkanKartu(1). K memainkan kartu: hitam-mimic. Menelusuri riwayat permainan... Belum ada kartu aksi yang dimainkan sebelumnya. Kartu mimic berlaku seperti wild biasa! Pilih warna (merah, kuning, hijau, biru). merah. Warna berubah menjadi merah. Giliran N.</pre>

Skenario tidak valid

Input Warna Salah:

Contoh Program
<pre> ?- mainkanKartu(6). K memainkan kartu: hitam-mimic. Menelusuri riwayat permainan... Belum ada kartu aksi yang dimainkan sebelumnya. Kartu mimic berlaku seperti wild biasa! Pilih warna (merah, kuning, hijau, biru). ungu. Warna tidak valid! Pilih kembali. Pilih warna (merah, kuning, hijau, biru). hijau. Warna berubah menjadi hijau. Giliran N.</pre>

Dampak terhadap state permainan

- efekMimic membaca state dinamis kartuAksiTerakhir dan counterGiliran untuk kalkulasi selisih giliran, namun tidak mengubahnya. Predikat ini memicu perubahan state secara tidak langsung dengan melempar eksekusi lanjutan ke predikat eksekusiEfek/1.
- pilihWarnaMimic menangkap input (read) dari pengguna, melakukan validasi logika list (isMember), lalu secara langsung memodifikasi dan memperbarui state warnaAktif menggunakan retract dan asserta.

t. turnamen

Mode Turnamen merupakan variasi permainan yang dapat dipilih ketika menjalankan perintah start. (startGame/0). Pemain akan diminta memilih antara mode klasik atau mode turnamen. Jika mode turnamen dipilih, permainan tidak langsung dimainkan secara individu, melainkan dimulai dengan sistem tim 2 vs 2.

start.

Tahap ini merupakan proses inisialisasi permainan mode turnamen. Program akan menawarkan dua pilihan mode, dan jika mode turnamen dipilih, program meminta 4

nama pemain, membentuk dua tim secara acak, mendistribusikan kartu awal, serta menetapkan giliran pertama.

Contoh Program

```
| ?- start.  
Tersedia 2 mode permainan.  
1. Mode klasik  
2. Mode turnamen  
Pilih mode permainan: 2.  
  
Permainan dimulai dalam mode turnamen.  
  
Masukkan nama pemain 1: 'Ghina'.  
Masukkan nama pemain 2: 'Sahla'.  
Masukkan nama pemain 3: 'Naya'.  
Masukkan nama pemain 4: 'Kate'.  
  
Membentuk tim secara acak...  
  
Tim 1 : Ghina, Sahla  
Tim 2 : Naya, Kate  
  
Urutan pemain: Ghina - Naya - Sahla - Kate.  
  
Setiap pemain mendapatkan 7 kartu acak.  
  
Kartu discard top: hijau-7  
  
Giliran Ghina.  
  
true ?
```

Pemain

Mode turnamen selalu menggunakan tepat 4 pemain. Program meminta nama pemain satu per satu, memvalidasi keunikan dan format nama (harus diawali huruf kapital, berupa atom, tidak boleh duplikat). Setelah 4 nama valid terkumpul, program membentuk dua tim secara acak (masing-masing 2 pemain) dan menetapkan urutan bermain yang berseling antar tim.

Predikat `startGame/0` memanggil `inputNamaPemain(4, [], Daftar)` untuk mengumpulkan 4 nama, lalu memanggil `inisialisasiGameTurnamen/1`. Di dalamnya, `bentukTim/1` mengacak daftar pemain dan menetapkan fakta `tim1/1` dan `tim2/1`. Urutan pemain dibuat berseling antar tim agar giliran bergantian. Fakta `modeTurnamen/0` di-assert untuk menandai mode aktif.

Dampak terhadap state permainan

Menambahkan fakta-fakta: `modeTurnamen`, `tim1([P1,P2])`, `tim2([P3,P4])`, `urutanPemain/1`, `giliranSekarang/1`, `kartuTangan/2` untuk setiap pemain, `kartuTeratas/1`, `warnaAktif/1`, `arahPermainan/1`, dan `sisaDeck/1`.

swapKartu (<NomorUrutKartuKu>, <NomorUrutKartuTeman>) .

Aksi utama eksklusif mode turnamen yang memungkinkan pemain aktif menukar satu kartunya dengan satu kartu milik teman setimnya berdasarkan nomor urut. Berhasil dieksekusi akan langsung memindahkan giliran ke pemain berikutnya.

Skenario valid:

Contoh Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. biru-4 2. hijau-8 3. hijau-0 4. hitam-wild 5. hijau-reverse 6. kuning-skip 7. merah-9 Berikut kartu yang teman satu tim anda miliki (Sahla). 1. merah-8 2. kuning-1 3. hitam-mimic 4. hijau-6 5. merah-7 6. merah-0 7. merah-1 (79 ms) yes ?- swapKartu(1,6). Ghina menukar biru-4 dengan kartu merah-0 milik Sahla. Pertukaran kartu berhasil. Giliran Naya. true ? yes</pre>

Skenario tidak valid (indeks salah):

Contoh Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. biru-reverse 2. merah-skip 3. hijau-9 4. kuning-7 5. merah-1 6. merah-5 7. hijau-2 Berikut kartu yang teman satu tim anda miliki (Kate). 1. hijau-4 2. biru-3 3. biru-5 4. biru-7 5. biru-7 6. kuning-2 7. merah-reverse</pre>

```
(172 ms) yes
| ?- swapKartu(1,8).
Nomor kartu 8 milik Kate di luar batas (Kate punya 7 kartu).
swapKartu gagal. Periksa kembali nomor kartu dan kondisi permainan.

(31 ms) yes
```

Skenario tidak valid (salah satu pemain hanya memiliki 1 kartu):

Contoh Program
<pre> ?- lihatKartu. Berikut kartu yang anda miliki. 1. biru-6 Berikut kartu yang teman satu tim anda miliki (Kate). 1. hijau-8 2. merah-4 3. kuning-drawTwo 4. biru-7 5. biru-7 6. merah-reverse 7. hijau-drawTwo (15 ms) yes ?- swapKartu(1,2). Tidak bisa swap: kartumu hanya tersisa 1. swapKartu gagal. Periksa kembali nomor kartu dan kondisi permainan. yes</pre>

Skenario tidak valid (bukan mode turnamen):

Contoh Program
<pre> ?- cekInfo. Kartu discard top: biru-9. Urutan pemain: Naya - Sahla - Ghina. Nama pemain 1: Naya Jumlah kartu : 12 Nama pemain 2: Sahla Jumlah kartu : 17 Nama pemain 3: Ghina Jumlah kartu : 13 (15 ms) yes ?- swapKartu(2,1). Perintah swapKartu hanya tersedia di mode turnamen. yes</pre>

Kegunaan

Predikat `swapKartu/2` menerima dua argumen berupa indeks kartu (1-based). Program memvalidasi secara berurutan: (1) permainan dalam mode turnamen, (2) pemain belum melakukan swap di giliran ini (cek fakta `sudahSwap/0`), (3) indeks tidak melebihi jumlah kartu masing-masing pemain, (4) kedua pemain memiliki lebih dari 1 kartu. Jika semua valid, kartu ditukar, fakta `sudahSwap` di-assert, lalu `gantiGiliran/0` dipanggil karena `swapKartu` dikategorikan sebagai Aksi Utama.

Dampak terhadap state permainan

Memperbarui `kartuTangan(Pemain, _)` dan `kartuTangan(Teman, _)` dengan tangan baru hasil pertukaran. Menambahkan fakta `sudahSwap` (diretract saat `gantiGiliran` dipanggil). Memperbarui `giliranSekarang/1` ke pemain berikutnya.

B. PEMBAGIAN DAN PERSENTASE KERJA ANGGOTA

No.	Nama (NIM)	Bagian	Persentase
1.	Ghina Emelia Yantes (13525119)	Milestone 1: - startGame (inisialisasi permainan: pemain dan distribusi awal kartu) - Integrasi fitur dan debugging Milestone 2: - Main - Integrasi fitur dan debugging Milestone 3: - God's Hand (bonus) - Mode turnamen - Integrasi fitur dan debugging	31%
2.	Sahla Nailah Salsabilla (13525134)	Milestone 1: - kartu.pl (inisialisasi bentuk kartu dan efek kartu aksi) Milestone 2: - cekInfo -antang Milestone 3: - saveGame - Mimic Card (bonus)	23%
3.	Cathrine Angel Siburian (13525138)	Milestone 1: - lihatCommand - lihatKartu Milestone 2: - sembunyikanKartu(bonus) - endGame Milestone 3: - update sembunyikanKartu - update endGame	23%
4.	Nayla Putri Ghaisani (13525140)	Milestone 1: - mainkanKartu - ambilKartu Milestone 2: - aturan uni - aturan tangkap milestone 3: - loadGame - Mimic Card (bonus)	23%

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM	: Ghina Emelia Yantes/13525119 Sahla Nailah Salsabilla/13525134 Cathrine Angel Siburian/13525138 Nayla Putri Ghaisani/13525140
Fakultas/Sekolah	: STEI-K
Kode Mata Kuliah	: IF1221
Nama Mata Kuliah	: Logika Komputasional
Judul Tugas/Proposal	: Praktikum IF1221 Logika Komputasional 2025/2026

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya /Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya /Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya/ Tidak *	Membantu mengidentifikasi kesalahan syntax dan mengembangkan ide algoritma agar lebih efisien.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya /Tidak*	
Lainnya, Jelaskan:	-	-

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 31 Mei 2026



Ghina Emelia Yantes
13525119